06 - DOCUMENTAÇÃO TÉCNICA DO MVP (PARTE 1)

Disciplina: Projeto Aplicado I
Empresa Parceira: Schulz S.A. (Laboratório de Metrologia)
Professor Orientador: MSc. Hugo Menezes Barra
Grupo: Fabiano Dalmolin, Felipe Sancho, Gabriel Araldi, Kauan Ramalho, Roberto Garcia
Link GitHub: Clique aqui para acessar o repositório

---

1. Introdução e Objetivo

Este documento apresenta a implementação construção efetiva do código do sistema. O objetivo central é implementar a arquitetura em Programação Orientada a Objetos (POO) em linguagem Python, construindo as classes de domínio, os objetos de acesso a dados e os algoritmos de validação de regras de negócio metrológicas e manipulação do banco relacional SQLite

O sistema foi desenvolvido para resolver o problema do Laboratório de Metrologia da Schulz S.A., onde os dados de medição gerados pela máquina tridimensional (MMC via software Zeiss Calypso) precisavam ser cruzados manualmente com planilhas eletrônicas. A solução atual realiza o cálculo metrológico de tolerâncias e o vínculo entre as tabelas de forma automática.

---

2. Estrutura de Programação Orientada a Objetos (POO)

O código fonte foi organizado no arquivo `classes_04.py` aplicando os seguintes conceitos:

- **Abstração e Classes de Domínio:** As entidades do processo foram mapeadas em três classes principais: `Tecnico`, `Equipamento` e `Medicao`, as quais possuem seus próprios atributos e métodos específicos.
- **Encapsulamento e Regra de Negócio:** A lógica de validação das medições foi inserida dentro do construtor da classe `Medicao`. O sistema executa o método interno `processar_regras_metrologicas()`, que calcula a média aritmética das duas passagens da máquina e valida o resultado contra os limites superior e inferior estabelecidos pela engenharia. O status de aprovação é gerado automaticamente.
- **Divisão de Responsabilidades:** A classe `DatabaseManager` ficou responsável exclusivamente pela infraestrutura do banco de dados (abrir conexão, fechar conexão, executar comandos e ativar chaves estrangeiras com `PRAGMA foreign_keys = ON;`), mantendo as classes de domínio separadas da persistência.

---

3. Diagrama de Classes (UML)

classDiagram class DatabaseManager { +str db_name +conectar() +desconectar() +inicializar_banco() }

```
class Tecnico {
    +int id_tecnico
    +str nome
```

```
    +str matricula
    +salvar(db_manager)
}

class Equipamento {
    +int id_equipamento
    +str codigo_tag
    +str denominacao
    +str fabricante
    +str numero_serie
    +str numero_certificado_externo
    +date data_ultima_calibracao
    +date data_proxima_calibracao
    +salvar(db_manager)
}

class Medicao {
    +int id_medicao
    +int id_tecnico
    +int id_equipamento
    +str measurement_plan
    +str part_number
    +str nome_caracteristica
    +date data_medicao
    +str hora_medicao
    +float valor_nominal
    +float tolerancia_superior
    +float_tolerancia_inferior
    +float medicao_1
    +float medicao_2
    +float media_calculada
    +str status_aprovacao
    +processar_regras_metrologicas()
    +salvar(db_manager)
}

Medicao --> Tecnico : "id_tecnico (FK)"
Medicao --> Equipamento : "id_equipamento (FK)"
```

4. Tratamento de Erros e Regras do Banco

Para garantir a consistência das informações e evitar falhas no armazenamento, foram implementadas as seguintes rotinas:

1. **Tratamento de Exceções:** Os métodos de inserção e consulta utilizam blocos `try-except` para capturar erros do tipo `sqlite3.Error`. Isso evita o fechamento inesperado do programa caso ocorra alguma falha no banco de dados.
2. **Integridade Referencial:** A configuração `ON DELETE RESTRICT ON UPDATE CASCADE` impede que registros fiquem órfãos. O SQLite bloqueia a exclusão de um técnico ou equipamento que possua medições associadas, garantindo o histórico para auditorias de qualidade.

3. **Restrição de Unicidade:** O campo `codigo_tag` na tabela de equipamentos foi definido como `UNIQUE`, impedindo o cadastro duplicado do mesmo instrumento.

5. Teste de Validação

O arquivo `main_05.py` foi executado para validar os requisitos do sistema. O teste simulou a gravação de um técnico, de um equipamento e de duas medições distintas (uma com resultado dentro das tolerâncias e outra fora).

Por fim, o script executou uma consulta utilizando `INNER JOIN` para unificar os dados e exibir o relatório de calibração formatado diretamente no terminal, comprovando que o cruzamento de dados funciona sem a necessidade de planilhas manuais.

6. Repositório do Projeto

- **Link de Acesso:** Clique aqui para acessar o repositório

Nota de Execução e Reiniciação Automatizada:
Para garantir que o script de demonstração (`main_05.py`) possa ser executado repetidas vezes sem falhas, foi inserido uma rotina automatizada no início da inicialização que remove o arquivo local `metrologia_schulz.db` caso ele já exista. Essa prática limpa a massa de testes anterior, permitindo que as restrições de unicidade (`UNIQUE`) e integridade do banco de dados sejam validadas do zero a cada execução, sem gerar conflitos de dados duplicados durante a correção do trabalho.